Reconstructing Neuron: A Journey from Errors to Historical Rules

Legal Immigrants
Neural Wave AI Hackathon

***Abstract*—We reconstructed an undocumented insurance engine from exported histories and reference outputs. Error analysis, decision trees and controlled experiments exposed historical rounding, calendar conventions and hidden accounting state. We encoded these findings in a deterministic replay, then extended the investigation to a surprise tax task with sparse exact-row feedback. This report emphasizes the discovery process and its limits; evaluation results remain provisional pending another run.**

I. Starting with observations

The original tasks covered commissions, premium due dates, maturity values, surrender values, annual statements and suspected manual corrections. We had contract histories, bookings, rates and reference outputs, but no complete executable specification. The objective was to reproduce what Neuron did, including historical conventions, with exact row agreement.

Through upload #26, our focus was on the original tasks; later work concentrated on the surprise tax task. Several implementations developed in parallel. Upload numbers mark stages of investigation, not isolated experiments on one continuous code line.

II. Turning mismatches into hypotheses

Reasonable formulas worked unevenly. Instead of treating every residual as noise, we grouped errors by date, tariff and intermediate accounting state. Decision trees helped locate informative subgroups. Formula searches and manual inspection then supplied competing explanations.

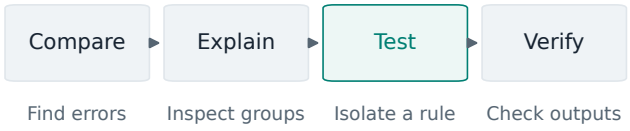


Fig. 1. Our discovery loop. Trees and searches suggested hypotheses; deterministic rules produced the final outputs.

A final balance alone did not reveal the cause. Comparing intermediate premiums, charges and allocations exposed where the first divergence occurred. Each explanation needed a focused check before becoming a reusable rule.

III. Discoveries that changed the model

A. One historical boundary, different arithmetic

Historical evidence connected a system conversion to 11 August 2007. COBOL-era calculations truncate where Java-era calculations round half-up. The applicable date depends on the operation, rather than a universal policy label.

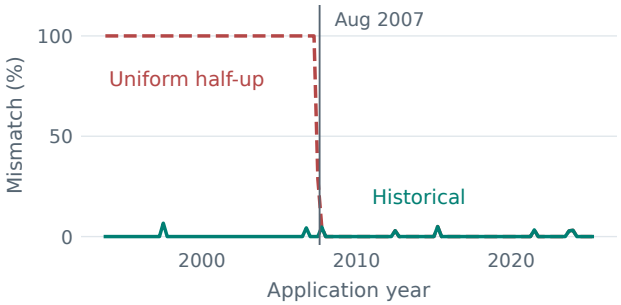


Fig. 2. Measured t2 training mismatch by application quarter. Only rounding was changed; rates and age conventions stayed fixed. Event 800 supplied the historical boundary.

Rounding counterfactual	Exact matches / 3,019
Uniform half-up	2,184
Uniform truncation	828
Historical conventions	**3,011**

B. The export did not contain all relevant state

A narrow birthday subgroup motivated testing days-based versus months-based entry ages. Calendar investigation separated due dates, historical anchors and collection dates. Accounting required another step: reconstructing holdings absent from individual charge rows.

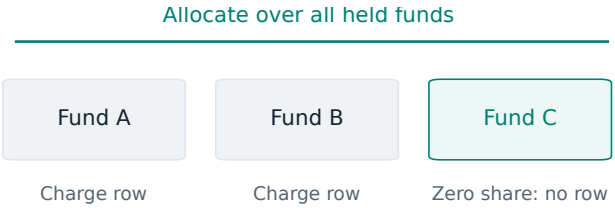


Fig. 3. Conceptual hidden-holdings example. A charge share rounding to zero can leave no exported charge row, yet the holding still affects allocation.

IV. The solution: replay historical state

The selected implementation combines explicit historical rules with shared contract and transaction state. It uses decimal arithmetic, the appropriate historical rates and date conventions, and prices and currency conversion to reconstruct financial events.

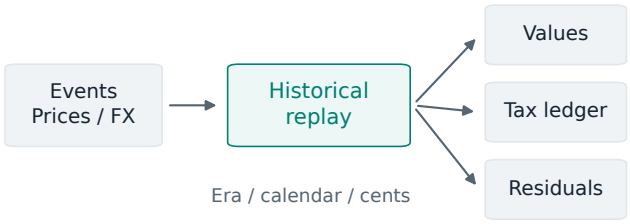


Fig. 4. Conceptual architecture of the chosen solution. Shared replay supports value reconstruction, tax-ledger calculations and residual checks; it is not a complete call graph.

Sharing computation also addressed execution constraints. An early annual-statement evaluation timed out after surrender calculations. Historical records describe later reuse and data-access improvements; timings from different versions and machines do not establish a controlled speedup.

After systematic fixes, whole-currency residuals suggested manual interventions: 38 of 54 flagged rows were associated with listed exceptions; 16 remained suspected edits. The number of flags is not a verified anomaly-detection score.

V. The surprise tax task

The tax extension initially lacked exact amount labels in the booking export. Booking presence helped identify applicability, but did not prove the base, amount or deadline. Two matching published examples provided little coverage.

Later investigations examined historical rate periods, premium costs already returned by earlier payouts, correction rounding and remittance calendars. The selected implementation carries this state forward in a tax ledger. Some annual taxes use effective years; others use remittance years.

A. Correctness also needed usable engineering

A correct formula still needed to run within the evaluation budget and tolerate incomplete exports. Historical records describe readiness checks for normal execution, resource limits, invariants, alternative dates and damaged inputs. These checks complemented exact-output comparisons by testing how the program behaved beyond the ordinary training run.

Those records belong to specific versions. They do not establish that every tax rule or the latest selected code passed an independent robustness test. Separating functional evidence, engineering evidence and unresolved cases kept a successful local run from becoming a broader claim than it supported.

VI. Learning from unsuccessful experiments

A payout row scores only when all six fields match. Correcting one of two wrong fields can therefore leave the score unchanged. No score movement cannot reject a hypothesis: another error may mask a fix, affected cases may be absent, or gains and losses may cancel.

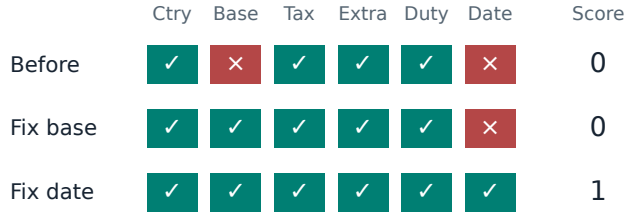


Fig. 5. Invented correctness pattern for a six-field payout row. Extra denotes surcharges; Duty denotes stamp duty. A valid partial fix may earn no point.

We also compared output columns to isolate bundled changes. Turning the upload-#53 calendar adjustment on and off moved 150 of 2,493 payout dates, while all other fields and annual rows stayed identical.



Fig. 6. Measured local calendar toggle on older #53 code. It cannot, by itself, explain an annual-score improvement between uploads. It does not establish calendar correctness.

A. What the weak experiments taught us

An upload could bundle several reversions and new rules. Its aggregate score did not identify which change helped. We therefore compared source versions, affected rows and changed columns before assigning an explanation to a result.

Some candidate rules also produced identical outputs on the available export. That agreement could not distinguish the hypotheses: the cases where their predictions differed were missing. A booking-presence proxy likewise could not verify an exact tax amount. These limitations motivated explicit diagnostics and narrower conclusions instead of treating an unchanged score as a rejection.

VII. Historical mechanisms in the tax solution

A. Rates belonged to periods, not just the final date

The selected implementation splits holdings at known tax-rate boundaries and calculates the gain attributable to each period. Its structure is

$$T = \sum_k r_k \, \Delta G_k,$$

where period gains are derived from values and premiums before the applicable payout share and tax rounding. This differs from applying the final rate to the entire gain.

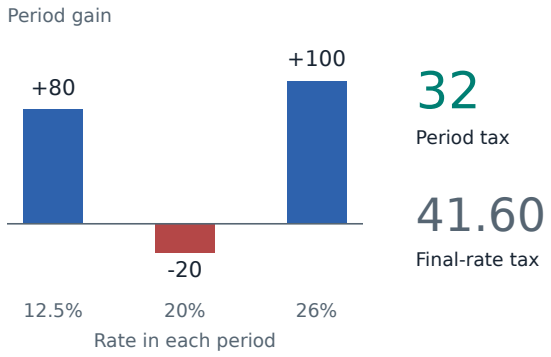


Fig. 7. Illustrative arithmetic using invented gain units. Period contributions are $10 - 4 + 26 = 32$; applying 26% to the total gain gives 41.60. This explains the implemented mechanism, not an independent validation of the rule.

B. Earlier payouts changed later calculations

Returned premium costs needed to be rounded and carried forward in the relevant currency. A later withdrawal therefore depended on previous withdrawals, rather than only the current payout. Correction bookings similarly required attention to whether rounding happened on each component or once on the aggregate adjustment.

C. An event could belong to different tax years

The financial event date, effective date and remittance date were distinct. The chosen annual calculation assigns some stamp taxes by effective year, while withholding and surcharges use the remittance year. This is why a calendar change and an annual-tax change must be investigated separately.

Together, these mechanisms extended the same central idea as the original tasks: recover historical state and operation order before calculating the final row. Their implementation is inspectable; the available sample coverage still limits how strongly each rule can be described as recovered behavior.

VIII. Results in their evaluation context

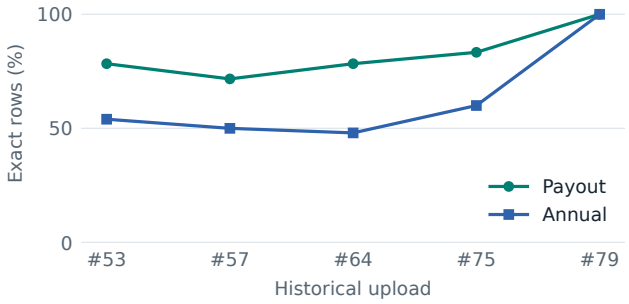


Fig. 8. Retrospective local fit on the same released 60 payout and 50 annual rows. Uploads are categories, not elapsed time. The samples supported rule development; the chart is not a live leaderboard or a held-out generalization curve.

Evaluation context	Payout	Annual
#79 released samples, local	60/60	50/50
#82 third set, public	9/40	27/30

The released samples were used during development. The latest screenshot also shows the original five scored task groups at 100%, but tax evaluation has moved to a third target set. These contexts cannot form a fixed-test learning curve. Another run is planned; the screenshot-to-commit mapping remains unverified.

IX. Collaboration and lessons

Humans chose the direction, interpreted feedback and selected the solution. AI helped explore code, propose formulas, implement diagnostics and document findings. Source inspection, controlled comparisons and evaluation artifacts checked those suggestions. Historical notes also record fabricated local-model results, which reinforced the need for executable evidence rather than fluent explanations.

The strongest lesson was to turn a mismatch into a discriminating experiment. Trees localized patterns, intermediate states localized causes, and controlled changes narrowed conclusions. Shared replay then turned the discoveries into a consistent implementation across related calculations.

Our result is an explicit historical reconstruction. Unseen tax payout behavior remains the main limitation. The next evaluation can update that evidence without changing the story of how we investigated it.

Evidence and reproducibility

Figure data, rendering scripts and source mappings accompany the report. Rounding was measured at commit `8247f0f`; the calendar toggle used older upload-#53 code. The companion README distinguishes recorded historical results, local experiments and screenshot evidence. No new complete solution-validation suite was run for this report revision.